AlphaLab — Full Project Context

Purpose of this document: Give any new AI agent or engineer complete context on AlphaLab — what it is, why it exists in its current form, what was cut and why, and every architectural decision made — without needing prior conversation history.

---

1. What AlphaLab Is

AlphaLab is a research platform that stress-tests predictive alpha factors (initially on NIFTY 50 equities) to determine whether they are genuinely robust or merely overfit to historical data.

Reframing (important): AlphaLab should not be pitched or built as "a quant project." It should be understood as:

> A research platform for evaluating whether predictive signals remain reliable under realistic data degradation.

Finance is the application domain, not the core identity. This framing makes the project legible to ML, research, and infrastructure audiences, not only quant-finance ones. It also fits a broader portfolio pattern the author is building — projects that evaluate systems rather than just building them (e.g. a separate project evaluating model behavior, another evaluating data quality). AlphaLab's role in that pattern: **evaluating predictive signal robustness.**

---

2. The One Research Thesis

Everything in this project must serve exactly one question:

> **Can we distinguish genuinely robust predictive factors from overfit ones using systematic stress testing?**

Any proposed feature should be tested against: *"Does this help answer the thesis?"* If not, it is deferred, not built.

The original project brief (AlphaLab) was far broader — it proposed 7 engines, LLM-assisted factor generation, regime detection, cross-market transfer, factor decay tracking, SHAP explainability, MLflow, multi-tier Celery queues, and a microservice-style monorepo. That version was rejected as too vast for a portfolio project: it read as a startup roadmap rather than something one or two people could finish and defend line-by-line in an interview. The project was deliberately trimmed to the single thesis above, with everything else pushed to a documented "deferred" list (see Section 9).

---

3. History of Key Decisions (why the project looks the way it does)

- **Original scope rejected:** too many engines, tools listed for signal rather than necessity, no single finishable core.
- **Trimming pass:** kept only what serves the robustness thesis — DSL, backtesting, robustness engine (noise + missing data only), simple leaderboard. Cut LLM generation, regime detection, decay tracking, SHAP, MLflow, multi-queue Celery, monorepo package explosion.
- **Market choice:** India (NIFTY 50) over US, deliberately — less saturated as a portfolio story, aligns with the author's placement context, and Yahoo Finance covers NIFTY tickers (`.NS` suffix) well enough to avoid NSE's own data-access friction.
- **Second review pass (external, treated as authoritative refinement, not a scope change):** confirmed the trimmed direction was correct and should not be expanded again, but recommended re-adding a small number of structural elements that increase engineering signal without increasing scope:
  - Market Data Provider abstraction (interface + Yahoo implementation) instead of direct coupling to `yfinance`
  - Universe as a first-class abstraction (not a hardcoded NIFTY50 list)
  - Automatic research report generation per experiment
  - ADRs (Architecture Decision Records) for every major decision
  - Learning notes per feature (problem, solution, alternatives rejected, tradeoffs)
  - "Interview Defense" sections per major component
  - Failure-reasoning output from the robustness engine (not just a score)
  - Keep logical package boundaries (`api/ worker/ dsl/ engine/ data/ web/`) — this is maintainability, not microservices, and should not be collapsed into one giant folder
- **Documentation policy adopted:** the codebase is not the sole source of truth; documentation is a first-class, continuously updated artifact (see Section 10).

---

4. Non-Goals (things AlphaLab is explicitly NOT trying to be)

- Not a production trading system
- Not a multi-market platform in v1 (India only; cross-market is future work)
- Not a demonstration of every tool the author knows — tool inclusion for its own sake is explicitly rejected
- Not built for scale it doesn't have (no Kubernetes, no multi-worker Celery, no distributed execution, no enterprise data vendors like Bloomberg)
- Not trying to prove multiple research questions at once (regime sensitivity, decay, cross-market generalization are separate theses, deliberately deferred)

Guiding question for any future addition (the project's "North Star"):

> Does this change make AlphaLab more valuable to a Staff Engineer or Research Engineer reviewing the work, or does it simply make the architecture bigger? If "bigger," don't build it. If "more insightful, more defensible, or more rigorous," it's worth doing.

---

5. System Architecture

```text
Next.js Frontend
      ↓
FastAPI Backend  ──────────────→ Celery Worker (Redis broker, single queue)
      ↓                                ↓
  Factor DSL Compiler          Backtest / Robustness Jobs
      ↓                                ↓
PostgreSQL (metadata)          DuckDB (market data, factor values, results)
```

Repository structure (flat, logically bounded, not microservices):

```text
alphalab/
├── api/              # FastAPI app: routes, schemas, auth
├── worker/           # Celery tasks: backtest, robustness jobs
├── web/              # Next.js frontend
├── dsl/              # Factor DSL grammar, parser, compiler
├── engine/           # factor evaluation, backtesting, robustness scoring
├── data/             # provider abstraction, ingestion, DuckDB access
├── tests/
├── docs/
│   ├── 00_MASTER_PLAN.md
│   ├── 01_ARCHITECTURE.md
│   ├── 02_CURRENT_STATE.md
│   ├── 03_NEXT_STAGE.md
│   ├── adr/
│   └── learning_notes/
└── infra/            # docker-compose, deploy configs
```

Design principle: **design for extension, implement for simplicity.** Example — architecture exposes `MarketDataProvider`, implementation is only `YahooProvider`. Architecture exposes `ExperimentTracker`, implementation is only `PostgresExperimentTracker`. The abstraction is real; the concrete implementation stays minimal until a second implementation is actually needed.

6. Technology Stack and Justifications

Layer	Choice	Justification
Frontend	Next.js + TailwindCSS + shadcn/ui	Fast to build, clean defaults
Visualization	Recharts	Equity curves, IC history, robustness heatmap
API	FastAPI	Async support, reused from author's prior project (APIDev)
Task Queue	Celery + Redis, single queue, single worker	Backtests take 30–60s; this is a measured latency problem, not a scaling exercise. Justification must remain concrete: "backtests took Xs, so we moved them off the request thread"
Metadata DB	PostgreSQL (Neon)	Users, experiments, factor metadata, job status
Analytical DB	DuckDB	Columnar, embedded, fast for rolling-window factor computation over OHLCV; deliberately separate from Postgres (relational metadata vs. analytical warehouse split is itself an interview-defensible decision)
Market Data	Provider abstraction (`MarketDataProvider` interface) → `YahooProvider` concrete implementation	Avoids direct coupling to `yfinance`; future providers (Polygon, Bloomberg) can be added without touching calling code
Auth	JWT (PyJWT)	Reused pattern from prior project
CI/CD	GitHub Actions → Vercel (web) + Render (api/worker) + Neon (db)	Lint → test → build, previously proven pattern

Explicitly excluded, with reasons:

- **MLflow** — a Postgres table (`factor_id, params, metrics, timestamp`) does the same job at this scale; could be swapped for an `MLflowProvider` later if ever needed
- **Multi-queue/priority Celery, multiple workers, distributed execution** — solves a scaling problem the project doesn't have

- **Microservice-style monorepo tooling** — architecture for a team, not a solo/two-person project; reads as over-engineering to reviewers
- **Sentry, structlog, dataset versioning infra** — no production traffic yet to justify observability overhead

---

7. Data Layer

Universe (first-class abstraction)

Not implemented as a hardcoded `NIFTY50` list. Built as a generic `Universe` concept, with `NIFTY50Universe` as the only concrete implementation for v1. Universe membership must be point-in-time ("constituents as of date X") to avoid survivorship bias in backtests — a known interview trap.

Source

Yahoo Finance, `.NS`-suffixed tickers (e.g. `RELIANCE.NS`), for daily OHLCV. NSE's own data feeds are avoided in v1 due to access friction (no clean REST API, captcha/rate-limit issues on bhavcopy scraping).

Pipeline

```text
YahooProvider (implements MarketDataProvider)
      ↓
Ingestion Job (Celery, scheduled)
      ↓
Validation Layer
   - missing bar detection
   - overnight price jump flagging (possible corporate action)
   - schema/type checks
      ↓
DuckDB: ohlcv table
      ↓
Factor Value Computation (on demand per experiment)
      ↓
DuckDB: factor_values table
```

Corporate Action Policy (must be explicitly documented, not left implicit)

Indian equity corporate actions (splits, bonuses, dividends) are noisier on Yahoo Finance than for US tickers. Policy: flag any overnight price move beyond a defined threshold (e.g. ±15%) for review rather than trusting adjusted close blindly. This policy must live in its own doc (`docs/corporate_actions.md` or equivalent section in architecture doc).

---

8. Core Components

8.1 Factor DSL (highest engineering-signal component — do not cut)

Users never write arbitrary Python. Rationale: safety (no `eval`, no `import os`), tractable leakage detection, and future LLM-compatibility (deferred but designed for).

Pipeline: `DSL string → Parser (AST) → Validator → Compiler → Executable factor function`

v1 primitives: `Momentum(n)`, `Volatility(n)`, `RollingMean(n)`, `RollingStd(n)`, arithmetic operators (`+ - * /`). No custom functions, no I/O, no imports.

Validator checks: syntax correctness, no forward-looking data (leakage), computational complexity bound.

8.2 Backtesting Engine

Method: walk-forward validation (expanding and/or rolling window) — never a single static train/test split, which fails due to temporal leakage and look-ahead bias. This distinction must have its own learning note; it's one of the easiest ways to fail a quant/ML interview if misunderstood.

Outputs: Sharpe, Sortino, Calmar, Information Coefficient (IC), Rank IC, Max Drawdown, Turnover.

8.3 Robustness Engine (the core thesis-proving component)

Exactly two stress tests in v1:

1. **Noise injection** — perturb price/volume by 0.5%, 1%, 2%, remeasure performance
2. **Missing data** — randomly drop 5%, 10%, 20% of observations, remeasure performance

```text
Robustness Score = Average Performance Under Stress / Original Performance
```

Failure reasoning (added per review): the engine should not output only a numeric score. It should also characterize *why* a factor failed — e.g. primarily noise-sensitive vs. primarily missing-data-sensitive, and under what conditions (e.g. high-volatility periods) instability concentrates. This can start as heuristic logic; it substantially increases the platform's interpretive value beyond a bare metric.

8.4 Research Report Generator (added per review — treat as a demo centerpiece)

Every completed experiment automatically produces a report containing: factor formula, backtest metrics, stress test results, robustness score, a recommendation, and reasons for

failure (if any). This serves simultaneously as an interview demo artifact, a research artifact, and a reproducibility artifact.

8.5 Leaderboard + Factor Detail Frontend

Two screens only in v1:

1. **Factor Leaderboard** — sortable by Sharpe, IC, RankIC, Robustness Score
2. **Factor Detail Page** — formula, metrics, equity curve, robustness heatmap (noise × missing-data grid), and the generated research report

No regime explorer, cross-market explorer, or multi-factor comparison screen unless the corresponding deferred feature is eventually built.

---

9. Deliberately Deferred Features (keep this list visible — do not delete)

Feature	Why deferred
Regime Detection (HMM/K-Means)	A thesis-sized ML project on its own; shallow implementation would invite unanswerable interview questions
Cross-Market Transfer (India ↔ US)	Doubles data pipeline scope; only meaningful once the India-only robustness thesis is proven
Factor Decay Tracking	Requires months of live data to be meaningful; cannot be demonstrated from historical backtests alone
SHAP Explainability	Not load-bearing for the robustness thesis; adds a dependency without proving the core claim
LLM-Assisted Factor Generation	Highest risk of looking like a thin wrapper around a prompt; dilutes the quant-rigor story the project is built to tell

Retaining this list explicitly (rather than deleting cut features) is itself treated as a signal of engineering maturity — it should appear in `docs/00_MASTER_PLAN.md`.

---

10. Documentation Policy (binding for all future work on this project)

The codebase is not the sole source of truth — documentation is a first-class artifact, updated continuously, such that a brand-new AI agent or engineer can recover full project context from exactly four files, with no prior conversation history required:

1. **`docs/00_MASTER_PLAN.md`** — long-term vision, research thesis, goals, roadmap, non-goals, success criteria. Changes rarely; treat as the project's constitution.

2. `docs/01_ARCHITECTURE.md` — living document describing how the system currently works: repo structure, package/service responsibilities, data flow, component diagram, database schema, APIs, worker architecture, DSL architecture, experiment pipeline, current tech choices, tradeoffs, future extension points.
3. `docs/02_CURRENT_STATE.md` — changelog/status, updated after every completed task: current phase/milestone, completed features, current architecture summary, DB state, APIs, testing status, documentation status, ADRs, known bugs, technical debt, open decisions, blockers, deferred features, what's working/not working, lessons learned, current repo tree.
4. `docs/03_NEXT_STAGE.md` — exactly one upcoming implementation milestone: objective, deliverables, files expected to change, dependencies, risks, acceptance criteria, interview concepts, learning objectives, estimated complexity, implementation order. Never contains future phases beyond the immediate next milestone; rewritten completely once that milestone is archived into `02_CURRENT_STATE.md`.

Workflow rule for every feature: implement → run tests → update architecture doc if needed → update current state → generate next stage. Only then is a task considered complete.

Master Plan Compliance rule: before implementing any feature, re-check it against `00_MASTER_PLAN.md`. If a requested feature contradicts the master plan or the single research thesis, do not implement it immediately — document the conflict, explain why, and propose whether to modify the roadmap, defer the feature, or reject it outright.

Architectural Discipline rule: never introduce a technology because it sounds impressive. Every dependency needs a concrete justification, documented as an ADR answering: why this, why not the alternatives, what tradeoffs are accepted.

Supporting artifacts to maintain alongside the four core docs:

- `docs/adr/` — one ADR per major decision (e.g. monorepo/flat structure, DuckDB choice, Factor DSL, factor storage schema, market data provider abstraction, Celery justification)
- `docs/learning_notes/` — per feature: what problem existed, why this solution, alternatives rejected, tradeoffs, future scaling, associated interview questions. Written like an internal engineering wiki, and treated as one of the project's strongest standalone assets.
- **Interview Defense sections** for major components (DSL, Celery, DuckDB, Robustness Engine, Backtesting Engine) — common interview questions, tradeoffs, scaling path for each.

Short version:

> Built AlphaLab, a robustness-aware factor research platform that stress-tests NIFTY 50 alpha factors against noise and missing data to distinguish stable signals from overfit ones.

Strong version:

> Designed and built a quant research platform with a custom DSL for safe factor definition (no arbitrary code execution), a walk-forward backtesting engine (Sharpe, IC, RankIC), and a robustness engine quantifying factor stability under noise and missing-data perturbation on NIFTY 50 equities. Introduced async task processing (Celery + Redis) after profiling backtest latency, offloading multi-second jobs from the request path.

Portfolio-level framing: AlphaLab should be described alongside the author's other evaluation-themed projects (one evaluating model behavior, another evaluating data quality, etc.) as part of a consistent narrative: the author builds *evaluation systems*, with AlphaLab applying that lens to predictive signal robustness in finance specifically.

Interview talking points to have ready:

- Why a DSL instead of sandboxed Python → safety model, leakage detection tractability
- Why DuckDB alongside Postgres → columnar analytical vs. relational metadata split
- Why Celery specifically, and only a single queue/worker → measured latency problem, not a checklist item
- Why walk-forward validation and not random train/test split → temporal leakage, look-ahead bias
- Why the Universe and Market Data Provider abstractions exist despite only one concrete implementation each → designed for extension, implemented for simplicity
- What was deferred and why → each deferred feature (regime detection, cross-market, decay, LLM generation, SHAP) is its own thesis; shipping any of them shallowly would have weakened the core robustness claim rather than strengthened the project

---

12. Summary for a New Agent Picking This Up

If given only this document: build a two-service system (FastAPI + Celery worker) backed by Postgres (metadata) and DuckDB (market data/factor values), with a custom DSL compiling safe factor expressions, a walk-forward backtesting engine, and a robustness engine that stress-tests factors under noise and missing data to produce a robustness score plus failure reasoning. Use provider/universe abstractions even though only one concrete implementation of each exists. Generate an automatic research report per experiment.

Expose exactly two frontend screens (leaderboard, factor detail). Do not add regime detection, cross-market transfer, decay tracking, SHAP, LLM generation, MLflow, or multi-worker Celery — these are deliberately deferred and documented, not omissions to silently fix. Maintain the four-file documentation structure continuously, and write an ADR for every non-trivial technical decision.